药材烘干过程传热传质模型的建立与数值求解

摘要

干燥是决定中药材成品品质的关键工序之一。针对圆柱形药材的热风烘干问题，本文建立了径向一维非稳态传热传质耦合模型，系统研究了预热平衡与恒温干燥两个阶段的温度场与水分浓度场演化规律，并据此确定了烘干时间。

针对问题一，基于附录 2 的常物性参数建立圆柱径向一维传热与水分扩散方程，采用显式有限差分法离散求解，得到 30 min 内药材内部的温度与水分浓度分布：1800 s 时中心温度 33.5743 °C、表面温度 36.7848 °C，表面水分浓度降至 1.4613 kg/kg，且网格加密验证表明结果已收敛。

针对问题二，将物性修正为随温度和水分浓度变化的附录 3 经验公式，建立守恒型变系数扩散方程，以两阶段烘房条件（前 2 h 按附件 1、之后恒温恒湿）驱动，采用守恒型有限体积显式格式求解，给出 3 h 内的温度与水分浓度分布；该模型将问题一的常物性模型推广为变物性形式，两问结果的差异与衔接在第 5 节说明。

针对问题三，为在 2∼3 天的长时间尺度上稳定、高效推进，改用隐式有限体积法（后向欧拉）配合 Thomas 追赶法求解，经空间网格加密与 Richardson 外推得到烘干结束时间约为 57.57 h（约 2.40 天），并通过时间步长加密与显式参考解对比验证了精度。

针对问题四，利用材料坐标 $\xi = r/R(t)$ 将失水收缩转化为等效扩散系数 $D/R^2(t)$，求得考虑收缩后烘干时间缩短至 51.12 h（约 2.13 天），较问题三缩短约 11.2%，并解释了其物理机理。

本文还对数值算法进行了系统比较：显式格式受稳定性条件 $\lambda_T = \alpha\Delta t/\Delta r^2 \le 0.5$ 约束，最大步长仅约 3.45 s；隐式格式无条件稳定，$\Delta t = 60$ s 时计算耗时约为显式格式的 1/40，而 Crank–Nicolson 格式兼具二阶精度与稳定性；Thomas 追赶法以 $O(N)$ 复杂度求解三对角方程组，在大规模时较直接求逆快约 5 倍。灵敏度分析表明，物性参数中烘干时间对水分扩散系数与药材半径最敏感，工艺参数中目标含水率的影响最大，而对流换热系数几乎不影响烘干时间，说明干燥过程由内部扩散控制；据此给出降低烘房湿度、适当提高温度、合理设定目标含水率等工艺优化建议。

关键字： 药材烘干　传热传质　圆柱径向扩散　有限体积法　Thomas 追赶法　灵敏度分析

目录

一、问题重述 ····· **4**

1.1 问题背景 ····· 4

1.2 问题的提出 ····· 4

二、模型假设与符号说明 ····· **4**

2.1 模型假设 ····· 4

2.2 符号说明 ····· 6

三、问题分析 ····· **7**

四、问题一的模型建立与求解 ····· **8**

4.1 控制方程与定解条件 ····· 8

4.2 显式有限差分离散 ····· 8

4.3 求解结果与分析 ····· 9

五、问题二的模型建立与求解 ····· **10**

5.1 变物性模型与两阶段烘房条件 ····· 10

5.2 守恒型有限体积显式格式 ····· 11

5.3 求解结果与分析 ····· 12

六、问题三的模型建立与求解 ····· **13**

6.1 隐式有限体积格式与 Thomas 追赶法 ····· 13

6.2 收敛性与精度校核 ····· 14

6.3 求解结果与分析 ····· 14

七、问题四的模型建立与求解 ····· **15**

7.1 材料坐标下的收缩模型 ····· 15

7.2 数值求解与结果 ····· 16

7.3 与问题三结果的对比分析 ····· 16

八、数值算法比较与选择 ····· **17**

8.1 显式与隐式格式的稳定性分析 ····· 18

8.2 线性方程组求解器的比较 ····· 18

8.3 算法选择结论 · · · · · 19
九、模型灵敏度分析 · · · · · **20**
9.1 灵敏度结论与工艺启示 · · · · · 20
十、模型评价与推广 · · · · · **23**
10.1模型优点 · · · · · 23
10.2模型缺点与改进方向 · · · · · 23
10.3模型推广 · · · · · 23
参考文献 · · · · · **23**
附录A 主要程序求解流程 · · · · · **24**
附录B 问题一求解程序（problem1.py） · · · · · **24**
附录C 问题二求解程序（problem2.py） · · · · · **26**
附录D 问题三求解程序（problem3.py） · · · · · **29**
附录E 问题四求解程序（problem4.py） · · · · · **31**

一、 问题重述

1.1 问题背景

干燥是中药材加工的关键工序之一，直接决定成品品质、干燥效率与能耗水平。热风烘干是最常见的干燥方式，通常包含**预热平衡**与**恒温干燥**两个阶段：预热平衡阶段烘房温湿度按预设曲线升温，药材随之升温并缓慢失水；恒温干燥阶段烘房保持恒温恒湿，药材水分持续蒸发直至达到含水率要求。传统试验优化方法成本高、周期长，因此有必要借助数理建模与数值仿真揭示干燥过程中的传热传质规律，为工艺参数优化提供理论依据。

某中药材呈圆柱形，长 25 cm、半径 2 cm。烘干开始时药材温度 28 °C，水分浓度（干基含水率）2.55 kg/kg，烘房温度与水分浓度随时间的实测变化见附件 1，烘干过程中药材半径随时间的收缩数据见附件 2。

1.2 问题的提出

问题 1：建立预热平衡阶段药材温度与水分浓度变化规律的数学模型（相关参数见附录 2），按表 1、表 2 格式给出 100、300、. . .、1800 s，到药材中心距离 0、0.5、1、1.5、2 cm 处的结果，并将完整结果保存到 `result1.xlsx`。

问题 2：建立整个烘干过程（预热平衡与恒温干燥）药材温度与水分浓度变化规律的数学模型（相关经验公式见附录 3），按表 3、表 4 格式给出 3 h 内每隔 0.5 h 的结果，并保存完整结果到 `result2.xlsx`。

问题 3：按烘干要求（药材各处水分浓度低于 0.15 kg/kg），确定烘干所需时间，按表 5 格式给出每隔 6 h 的水分浓度，并保存完整结果到 `result3.xlsx`。

问题 4：考虑烘干过程中药材因失水发生的尺寸变化（见附件 2），重新确定烘干时长（相关经验公式见附录 4），按表 6 格式给出结果，并保存完整结果到 `result4.xlsx`。

二、 模型假设与符号说明

2.1 模型假设

1. 药材为均匀、各向同性的圆柱体，长 25 cm 远大于半径 2 cm，忽略轴向传热传质，问题退化为径向一维问题；
2. 烘干开始时药材内部温度与水分浓度均匀分布；
3. 烘房内温度与湿度均匀，药材表面与热风之间按第三类边界条件换热换质，对流换热系数 h 与对流传质系数 h_m 为常数；
4. 忽略药材内部水分蒸发吸收的潜热及局部冷却效应，忽略辐射换热；
5. 水分在药材内部以扩散方式迁移（Fick 定律），干燥为准静态过程，无内部压力梯度；

6. 失水收缩是均匀的且仅沿径向发生（长度不变）：每个材料点保持其相对半径位置 $r/R(t)$ 不变，收缩规律由附件 2 给定；
7. 物性参数按附录 2∼4 的常数或经验公式取值；
8. 恒温干燥阶段烘房温度与水分浓度保持恒定，取附件 1 平稳段的稳定值（50 °C、0.04986 kg/kg）。

2.2 符号说明

符号	意义	单位
t	时间	s
r	到药材中心的径向距离	m
$R,\ R(t)$	药材半径（问题四中随时间收缩，由附件 2 给定）	m
T	药材温度	°C
T_{env}	烘房温度	°C
T_{set}	恒温干燥阶段烘房温度	°C
C	药材水分浓度（干基含水率）	kg/kg
C_{env}	烘房水分浓度	kg/kg
C_{set}	恒温干燥阶段烘房水分浓度	kg/kg
C^*	目标水分浓度（烘干合格标准）	kg/kg
ρ	密度	kg/m^3
c_p	比热容	$\text{J/(kg}\cdot\text{K)}$
k	导热系数	$\text{W/(m}\cdot\text{K)}$
h	对流换热系数	$\text{W/(m}^2\cdot\text{K)}$
h_m	对流传质系数	m/s
D	水分扩散系数	m^2/s
D_0	水分扩散系数经验公式的前置因子	m^2/s
α	热扩散率，$\alpha = k/(\rho c_p)$	m^2/s
$\lambda_T,\ \lambda_D$	温度、水分方程的稳定性参数（$\alpha\Delta t/\Delta r^2$、$D\Delta t/\Delta r^2$）	—
ξ	材料坐标，$\xi = r/R(t)$	—
$\Delta t,\ \Delta r$	时间、空间步长	s, m
t_f	烘干结束时间	h

三、 问题分析

烘干过程本质上是圆柱体内的非稳态导热与水分扩散耦合问题，控制方程为圆柱坐标下的径向扩散方程。四个问题在物理层次上逐级递进：

问题 1（预热平衡阶段，30 min）：ρ、c_p、k、h、h_m 按附录 2 取常数，水分扩散系数按附录 2 经验公式随 C 变化，边界条件由附件 1 的烘房温湿度曲线给定。这是典型的第二、三类边界条件下的圆柱非稳态扩散问题，可用显式有限差分法直接求解。

问题 2（整个烘干过程，3 h）：物性改为随 C、T 变化的附录 3 经验公式，方程变为变系数扩散方程；同时需要区分两个阶段：前 2 h 烘房条件按附件 1，之后进入恒温干燥阶段（50 °C、0.04986 kg/kg）。变系数方程宜写成守恒形式并采用有限体积法，以更好地保证通量守恒。

问题 3（确定烘干时间）：需要将问题 2 的模型推进约 2∼3 天。显式格式受稳定性条件限制必须取秒级小步长，约 20 万步，效率低下；应改用**无条件稳定**的隐式格式（后向欧拉），并用 Thomas 追赶法高效求解每步产生的三对角方程组，可将步长放大到 10∼60 s，并将径向网格加密至 $N = 320$ 以控制空间离散误差。

问题 4（考虑失水收缩）：药材半径由 2.000 cm 收缩至约 1.198 cm。若在物理坐标下求解，网格边界是移动的。引入**材料坐标** $\xi = r/R(t)$ 可将移动边界问题转化为固定区域问题：时间导数变换产生的对流项与材料点运动引起的对流项恰好抵消，收缩的全部影响归结为扩散算子乘以因子 $1/R^2(t)$，模型形式大为简化。

总体而言，四个问题在模型上层层递进而保持统一：问题一正是问题二在常物性下的特例（式 (9) 在物性取常数时退化为式 (1)、(2)）；问题三与问题二采用完全相同的物理模型，仅将模拟时域由 3 h 延长至烘干结束；问题四是问题三在材料坐标下的推广，收缩的全部影响归结为因子 $1/R^2(t)$。数值方法上，问题一为显式有限差分，问题二为守恒型显式有限体积（物性变化时更稳健），问题三、四为隐式有限体积，各格式均基于同一径向网格与相同的边界处理，仅在时间推进方式上按时间尺度需求选择显式或隐式。

总体求解思路如图 1 所示。

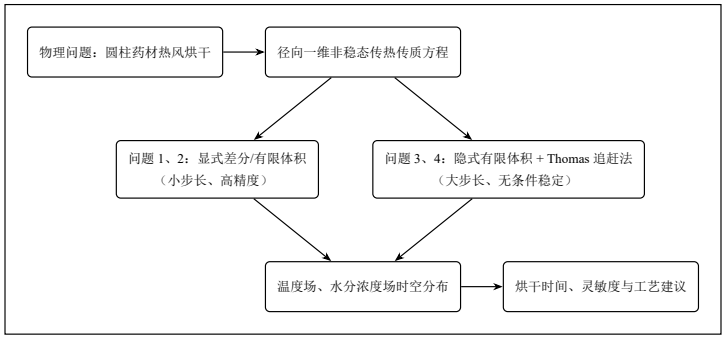


图 **1**　总体求解思路流程图

四、 问题一的模型建立与求解

4.1 控制方程与定解条件

预热平衡阶段物性按附录 2 取值，其中 $\rho = 820\,\mathrm{kg/m^3}$、$c_p = 2600\,\mathrm{J/(kg\cdot K)}$、$k = 0.36\,\mathrm{W/(m\cdot K)}$、$h = 25\,\mathrm{W/(m^2\cdot K)}$、$h_m = 8\times10^{-7}\,\mathrm{m/s}$ 为常数，水分扩散系数 D 随水分浓度变化，由式 (3) 计算。由于圆柱长 25 cm 远大于半径 2 cm，忽略轴向效应，温度场与水分浓度场只沿径向变化。温度场满足圆柱坐标下的径向一维非稳态导热方程

$$\rho c_p \frac{\partial T}{\partial t} = k\left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r}\frac{\partial T}{\partial r}\right), \qquad 0 < r < R,\ t > 0, \tag{1}$$

水分浓度场满足 Fick 扩散方程

$$\frac{\partial C}{\partial t} = D\left(\frac{\partial^2 C}{\partial r^2} + \frac{1}{r}\frac{\partial C}{\partial r}\right), \qquad 0 < r < R,\ t > 0, \tag{2}$$

其中水分扩散系数按附录 2 的经验公式

$$D = 7\times10^{-9}\,\mathrm{e}^{-0.89/C}\ \ (\mathrm{m^2/s}). \tag{3}$$

定解条件为：

- 初始条件：$T(r,0) = 28\,^\circ\mathrm{C}$，$C(r,0) = 2.55\,\mathrm{kg/kg}$；
- 中心对称条件：$\partial T/\partial r|_{r=0} = \partial C/\partial r|_{r=0} = 0$；
- 表面边界条件（第三类）：

$$-k\frac{\partial T}{\partial r}\Big|_{r=R} = h\big(T(R,t) - T_{\mathrm{env}}(t)\big), \qquad -D\frac{\partial C}{\partial r}\Big|_{r=R} = h_m\big(C(R,t) - C_{\mathrm{env}}(t)\big), \tag{4}$$

其中烘房温度 $T_{\mathrm{env}}(t)$ 与水分浓度 $C_{\mathrm{env}}(t)$ 由附件 1 线性插值到整数秒获得。

4.2 显式有限差分离散

将半径 $[0,R]$ 均匀剖分为 $N = 20$ 个区间，$\Delta r = 0.1\,\mathrm{cm} = 10^{-3}\,\mathrm{m}$，时间步长 $\Delta t = 1\,\mathrm{s}$。记 $T_i^n = T(i\Delta r, n\Delta t)$。对式 (1) 采用中心差分：

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha\left[\frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta r^2} + \frac{1}{r_i}\frac{T_{i+1}^n - T_{i-1}^n}{2\Delta r}\right], \tag{5}$$

其中 $\alpha = k/(\rho c_p) = 1.688\times10^{-7}\,\mathrm{m^2/s}$。中心点 $r = 0$ 处利用对称性 $T_{-1} = T_1$ 得 $T_0^{n+1} = T_0^n + 4\lambda_T(T_1^n - T_0^n)$，$\lambda_T = \alpha\Delta t/\Delta r^2 \approx 0.169$；表面节点 $r = R$ 引入虚拟节点 T_{N+1}、C_{N+1}，其值由式 (4) 的中心差分近似给出：

$$T_{N+1}^{n} = T_{N-1}^{n} - \frac{2\Delta r\, h}{k}\left(T_{N}^{n} - T_{\text{env}}^{n}\right), \qquad C_{N+1}^{n} = C_{N-1}^{n} - \frac{2\Delta r\, h_m}{D_{N}^{n}}\left(C_{N}^{n} - C_{\text{env}}^{n}\right), \tag{6}$$

再按与内部节点相同的差分模板计算表面值。水分浓度方程按同一格式离散，扩散系数 D 随各节点当前浓度 C_i^n 更新。稳定性条件 $\lambda_T, \lambda_D \le 1/2$ 显然满足（$\lambda_T \approx 0.169$，$\lambda_D = D\Delta t/\Delta r^2 \le 0.007$，因 $D \le 7\times10^{-9}\,\mathrm{m^2/s}$），格式稳定。

4.3 求解结果与分析

运行 1800 s（30 min），每 1 s、每 0.1 cm 输出完整结果至 `result1.xlsx`，论文所需数据见表 1 与表 2（即题目要求的表 1、表 2）。

图 2 和图 3 展示了 30 min 内温度与水分浓度的径向分布及时间演化。由图可见：药材温度自外向内逐步升高，1800 s 时中心 33.5743 °C、表面 36.7848 °C，仍低于烘房温度（此时 41.513 °C），说明预热尚未完成；水分仅在表层约 0.5 cm 范围内明显下降（表面降至 1.4613 kg/kg），中心仍保持 2.55 kg/kg，反映出水分扩散系数极小、干燥前沿推进缓慢的特征。由于温度扩散率远大于水分扩散率（$\alpha \gg D$），升温是”分钟级”过程，而干燥是”天级”过程，两者时间尺度分离。

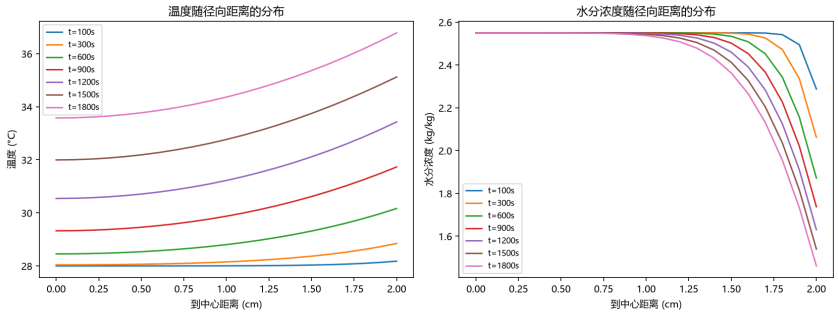


图 **2**　问题一：不同时刻温度（左）与水分浓度（右）的径向分布

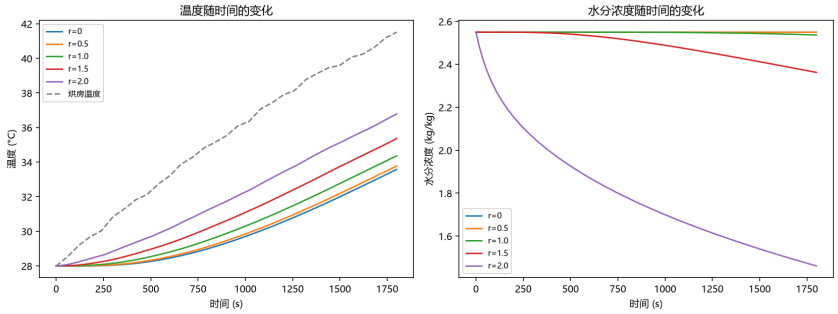


图 **3**　问题一：不同半径处温度（左）与水分浓度（右）随时间的变化

收敛性校核：将空间步长减半（$\Delta r = 0.05\,\mathrm{cm}$）重新计算，1800 s 时各点温度变化不超过 0.001 °C，表明结果已网格收敛。

表 **1**　　**30** 分钟内药材的温度（°**C**）

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0000	28.0003	28.0039	28.0321	28.1782
300	28.0405	28.0631	28.1507	28.3668	28.8464
600	28.4523	28.5350	28.8027	29.3144	30.1635
900	29.3230	29.4569	29.8741	30.6148	31.7291
1200	30.5414	30.7084	31.2210	32.1112	33.4263
1500	31.9944	32.1855	32.7648	33.7453	35.1200
1800	33.5743	33.7710	34.3632	35.3612	36.7848

表 **2**　　**30** 分钟内药材的水分浓度（**kg/kg**）

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2875
300	2.5500	2.5500	2.5500	2.5486	2.0617
600	2.5500	2.5500	2.5500	2.5337	1.8718
900	2.5500	2.5500	2.5495	2.5019	1.7376
1200	2.5500	2.5500	2.5477	2.4597	1.6305
1500	2.5500	2.5499	2.5437	2.4122	1.5401
1800	2.5500	2.5496	2.5369	2.3626	1.4613

五、 问题二的模型建立与求解

5.1 变物性模型与两阶段烘房条件

整个烘干过程中物性随水分浓度与温度显著变化，按附录 3 取：

$$\rho = 650 + 128C, \qquad c_p = 1450 + \frac{2736C}{C+1}, \qquad k = 0.21 + \frac{0.38C}{C+1}, \tag{7}$$

$$D = 2.4\times10^{-3}\,\mathrm{e}^{-0.45/C}\,\mathrm{e}^{-3850/T}\ \ (\mathrm{m^2/s},\,T\ \text{单位 K}),\tag{8}$$

由于 k、D 随 C 变化，传热与传质方程必须写成**守恒形式**：

$$\rho c_p\frac{\partial T}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left(rk\frac{\partial T}{\partial r}\right),\qquad \frac{\partial C}{\partial t}=\frac{1}{r}\frac{\partial}{\partial r}\left(rD\frac{\partial C}{\partial r}\right),\tag{9}$$

边界条件同式 (4)。烘房条件分两阶段处理：

$$\big(T_{\text{env}}(t),C_{\text{env}}(t)\big)=\begin{cases}\text{附件 1 插值}, & 0\le t\le 7200\,\text{s（预热平衡）},\\ (50\,^\circ\text{C},\ 0.04986\,\text{kg/kg}), & t>7200\,\text{s（恒温干燥）}.\end{cases}\tag{10}$$

依据：附件 1 中烘房温度在约 1.92 h 首次达到 49.9 °C，之后稳定在 50 °C，水分浓度稳定在 0.04986 kg/kg，故以 2 h 为阶段分界。

与问题一的衔接说明。问题二在同一时刻的结果与问题一存在小幅差异：$t=1800$ s 时，中心温度分别为 33.5743 °C（问题一，表 1）与 32.1879 °C（问题二，表 3），表面水分浓度分别为 1.4613 与 1.6483 kg/kg，而中心水分浓度分别为 2.5500 与 2.5499 kg/kg、几乎一致。该差异源于附录 2（常物性，$k=0.36\,\mathrm{W/(m\cdot K)}$，$D$ 为式 (3)）与附录 3（变物性，初始 $k=0.4830\,\mathrm{W/(m\cdot K)}$，$D$ 为式 (8)）对同一药材给出了不同的物性参数化，致使预热阶段的温升速率与表层干燥速率略有不同；水分场在未受干燥前沿影响的内部区域则几乎完全重合。从模型结构看，问题一恰是问题二的常物性特例（式 (9) 在物性取常数时退化为式 (1)、(2)），后续问题三、四均以问题二的变物性模型为基准，保证了四个问题在模型框架上的统一。

5.2 守恒型有限体积显式格式

对控制体积 $[r_{i-1/2},r_{i+1/2}]$ 积分式 (9)，得到守恒型显式格式。对内部节点（$1\le i\le N-1$）：

$$T_i^{n+1}=T_i^n+\frac{\Delta t}{\rho_i c_{p,i}\,i\Delta r^2}\Big[(i-\tfrac12)k_{i-1/2}(T_{i-1}-T_i)+(i+\tfrac12)k_{i+1/2}(T_{i+1}-T_i)\Big],\tag{11}$$

中心节点用对称性处理，表面控制体积（半宽 $\Delta r/2$）包含对流边界项：

$$T_N^{n+1}=T_N^n+\frac{2\Delta t}{\rho_N c_{p,N}\Delta r^2(N-\frac14)}\Big[k_{N-1/2}(N-\tfrac12)(T_{N-1}-T_N)-hR(T_N-T_{\text{env}})\Big],\tag{12}$$

其中界面导热系数取两侧节点值的算术平均 $k_{i+1/2}=(k_i+k_{i+1})/2$。水分浓度方程按同一格式离散，界面扩散系数同样取平均。该格式对变系数问题保持离散通量守恒，比

直接差分更稳健。取 $\Delta r = 0.1\,\mathrm{cm}$、$\Delta t = 1\,\mathrm{s}$。初始物性（$C = 2.55$）为 $\rho = 976.40\,\mathrm{kg/m^3}$、$c_p = 3415.3\,\mathrm{J/(kg\cdot K)}$、$k = 0.4830\,\mathrm{W/(m\cdot K)}$，热扩散率 $\alpha = 1.448 \times 10^{-7}\,\mathrm{m^2/s}$；全程水分扩散系数在约 $5.6 \times 10^{-9} \sim 1.26 \times 10^{-8}\,\mathrm{m^2/s}$ 之间（升温使 D 增大）。全程网格稳定性参数 $\lambda_T = \alpha\Delta t/\Delta r^2 \le 0.23$、$\lambda_D = D\Delta t/\Delta r^2 \le 0.013$，均满足显式格式的稳定性条件 $\lambda \le 1/2$。

5.3 求解结果与分析

运行 3 h，完整结果保存至 `result2.xlsx`，论文所需数据见表 3、表 4（即题目要求的表 3、表 4）。

表 **3**　　**3** 小时内药材的温度（°**C**）

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1879	32.3808	32.9647	33.9597	35.4128
1.0	40.3818	40.5538	41.0609	41.8789	42.9984
1.5	45.8477	45.9357	46.1939	46.6062	47.1417
2.0	48.4509	48.4888	48.5992	48.7743	49.0041
2.5	49.5472	49.5599	49.5969	49.6550	49.7301
3.0	49.8772	49.8807	49.8908	49.9068	49.9274

图 4、图 5 展示了 3 h 内温度与水分浓度的演化。3 h 时中心温度 49.8772 °C、表面 49.9274 °C，药材整体已与 50 °C 烘房基本达到热平衡；水分方面中心 1.7655 kg/kg、表面 1.0079 kg/kg，表面失水明显快于中心，干燥前沿自外向内推进，中心成为后续烘干的控制点。

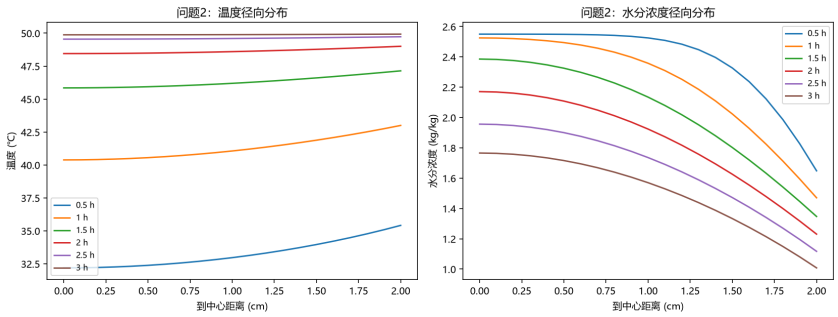


图 **4**　问题二：不同时刻温度（左）与水分浓度（右）的径向分布

表 **4**　　**3** 小时内药材的水分浓度（**kg/kg**）

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5487	2.5250	2.3267	1.6483
1.0	2.5250	2.4941	2.3578	2.0234	1.4704
1.5	2.3855	2.3253	2.1344	1.8020	1.3470
2.0	2.1707	2.1083	1.9236	1.6258	1.2306
2.5	1.9561	1.9001	1.7355	1.4718	1.1170
3.0	1.7655	1.7159	1.5698	1.3331	1.0079

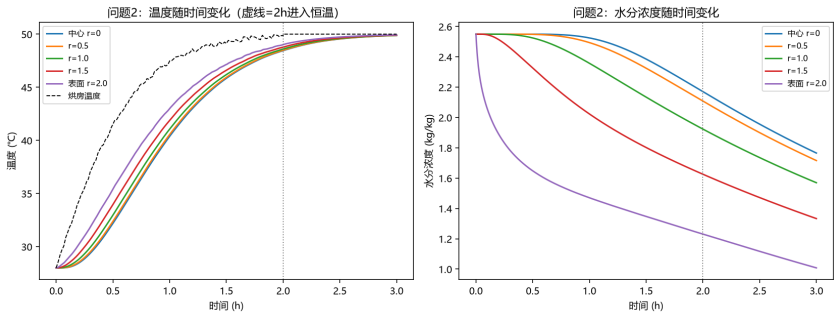


图 **5**　问题二：不同半径处温度（左）与水分浓度（右）随时间的变化（**2 h** 后进入恒温干燥）

收敛性校核：将空间步长依次减半（时间步长同步缩小），3 h 时表面水分浓度依次为 1.0079 → 1.0082 → 1.0082 kg/kg，结果已收敛。

六、 问题三的模型建立与求解

6.1 隐式有限体积格式与 Thomas 追赶法

问题三的物理模型与问题二完全相同，只是需要将模拟推进到烘干结束（各处水分浓度 $< 0.15\,\mathrm{kg/kg}$），时间跨度约 2~3 天。显式格式的稳定性条件 $\lambda_T = \alpha\Delta t/\Delta r^2 \le 0.5$ 要求 $\Delta t \le 3.45\,\mathrm{s}$，全程需约 20 万步，效率过低。为此改用**隐式（后向欧拉）格式：**对式 (9) 离散时空间项取新时间层，得到

$$\rho_i c_{p,i}\frac{T_i^{n+1}-T_i^{n}}{\Delta t} = \frac{(i-\tfrac{1}{2})k_{i-1/2}(T_{i-1}^{n+1}-T_i^{n+1}) + (i+\tfrac{1}{2})k_{i+1/2}(T_{i+1}^{n+1}-T_i^{n+1})}{i\Delta r^2}, \qquad (13)$$

每个时间步形成三对角线性方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{d}$（系数矩阵对角占优），由 **Thomas 追赶法**以 $O(N)$ 复杂度直接求解，水分浓度方程同理。隐式格式无条件稳定，可取 $\Delta t=10\,\mathrm{s}$（每 60 s 记录一帧）。为控制空间离散误差，径向网格加密至 $N=320$（$\Delta r=6.25\times10^{-5}\,\mathrm{m}$），空间收敛性详见 6.2 节。烘干结束判据取 $\max_i C_i<0.14995\,\mathrm{kg/kg}$（保证四位小数下严格小于 0.15）。

6.2 收敛性与精度校核

（1）**空间网格收敛性**：固定 $\Delta t=10\,\mathrm{s}$，径向网格依次取 $N=20/40/80/160/320$，烘干时间依次为 $56.52/57.05/57.32/57.43/57.50\,\mathrm{h}$，相邻增量 $0.53/0.27/0.12/0.07\,\mathrm{h}$ 逐次减半，呈一阶收敛；按 Richardson 外推得 $t_f\approx57.57\,\mathrm{h}$。烘干结束时刻是解的“病态泛函”：末段中心水分浓度仅以约 $0.0011\,\mathrm{kg/(kg\cdot h)}$ 的速度下降，$0.001\,\mathrm{kg/kg}$ 的空间离散误差即可使烘干时间偏移约 1 h，因此空间收敛性校核不可省略；

（2）**时间步长无关性**：$\Delta t=60/30/10\,\mathrm{s}$ 三次计算（$N=20$ 网格），烘干时间依次为 $56.483/56.479/56.469\,\mathrm{h}$，几乎不变，隐式格式对大步长表现出良好的精度；

（3）**与显式参考解对比**：在 3 h 处与问题二的显式解（$\Delta t=1\,\mathrm{s}$，同一 $\Delta r=0.1\,\mathrm{cm}$ 网格）对比，中心水分浓度 1.7659 对 1.7655、表面 1.0085 对 1.0079、中心温度 49.8746 对 49.8772 °C，偏差不超过 0.06%，说明隐式大步长与显式小步长的实现相互一致；

（4）**物质守恒校核**：以 $N=320$ 网格计算全过程含水量减少 $\Delta W=\int_V\big[C_0-C(r,t_f)\big]\mathrm{d}V=7.6209\times10^{-4}\,\mathrm{m^3}$，与表面累计对流通量 $F=\int_0^{t_f}h_m\big(C_s-C_{\mathrm{env}}\big)A\,\mathrm{d}t=7.6209\times10^{-4}\,\mathrm{m^3}$ 的相对偏差小于 0.1%，定量验证了守恒型格式的物质守恒性。

6.3 求解结果与分析

烘干结束时间取空间网格 Richardson 外推值 $t_f\approx57.57\,\mathrm{h}$（$N=320$ 直接解为 $207000\,\mathrm{s}=57.50\,\mathrm{h}$，两者相差约 4 min，工程上统一回答约 57.5~57.6 h，即约 2.40 天）。结束时中心水分浓度 $0.1499\,\mathrm{kg/kg}$、表面 $0.0525\,\mathrm{kg/kg}$，内部温度均已达 50.0 °C。表 5（题目要求的表 5）给出每 6 h、每 0.5 cm 的水分浓度，完整结果（每 60 s、每 0.1 cm）保存于 `result3.xlsx`。

图 6 展示了干燥全过程的水分浓度演化。物理规律分析如下：

- 表面干燥极快：6 h 时表面已降至 $0.53\,\mathrm{kg/kg}$，而内部只能依靠扩散缓慢跟上；
- 中心是烘干控制点：中心水分 18 h 后才降至 0.30，最终在 57.5 h 达到 0.15 以下；
- 干燥后期极度缓慢：$D\propto\mathrm{e}^{-0.45/C}$ 随 C 减小而急剧衰减，最后一段（C 从 0.17 降到 0.15）耗费约 16 h，这是总耗时接近 2.4 天的主要原因。

表 **5**　　药材烘干过程的水分浓度（**kg/kg**）

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0171	0.9882	0.9016	0.7551	0.5329
12	0.4562	0.4432	0.4031	0.3297	0.1641
18	0.2988	0.2915	0.2687	0.2253	0.0844
24	0.2378	0.2327	0.2167	0.1855	0.0664
30	0.2058	0.2018	0.1891	0.1640	0.0597
36	0.1858	0.1825	0.1718	0.1503	0.0565
42	0.1720	0.1691	0.1597	0.1406	0.0547
48	0.1618	0.1591	0.1506	0.1334	0.0536
54	0.1538	0.1514	0.1436	0.1276	0.0528
烘干结束 57.50	0.1499	0.1476	0.1401	0.1248	0.0525

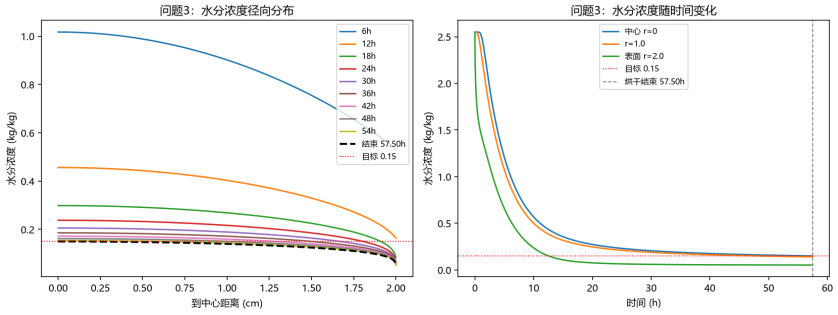


图 **6**　问题三：不同时刻水分浓度径向分布（左）与关键位置水分浓度随时间变化（右）

七、 问题四的模型建立与求解

7.1 材料坐标下的收缩模型

实际烘干中药材因失水收缩：附件 2 显示半径由 2.000 cm 单调收缩至约 1.198 cm（收缩约 40%），前 1 h 即收缩 10.3%，约 24 h 后基本稳定在 1.20 cm。物性按附录 4 取值：

$$\rho = 760 + 90C, \qquad c_p = 1850 + \frac{2150C}{C+1}, \qquad k = 0.12 + \frac{0.20C}{C+1}, \tag{14}$$

$$D = 4.2 \times 10^{-4}\, \mathrm{e}^{-0.30/C}\, \mathrm{e}^{-3850/T}. \tag{15}$$

引入材料坐标 $\xi = r/R(t)$。假定收缩均匀，即每个材料点保持相对位置 ξ 不变，其收缩速度 $v = \xi\,\mathrm{d}R/\mathrm{d}t$。将传热传质方程变换到 (ξ, t) 坐标系，时间导数变换产生的对流项与收缩对流项**恰好抵消**，方程化为简洁形式：

$$\left.\frac{\partial C}{\partial t}\right|_{\xi} = \frac{D}{R^2(t)}\left(\frac{\partial^2 C}{\partial \xi^2} + \frac{1}{\xi}\frac{\partial C}{\partial \xi}\right), \qquad \rho c_p \left.\frac{\partial T}{\partial t}\right|_{\xi} = \frac{k}{R^2(t)}\left(\frac{\partial^2 T}{\partial \xi^2} + \frac{1}{\xi}\frac{\partial T}{\partial \xi}\right). \tag{16}$$

即收缩的全部作用等价于把扩散算子乘以因子 $1/R^2(t)$：扩散路径随 R 缩短，干燥加速。移动表面 $\xi = 1$ 上的边界条件为

$$-D\left.\frac{\partial C}{\partial \xi}\right|_{\xi=1} = h_m R(t)\big(C - C_{\mathrm{env}}\big), \qquad -k\left.\frac{\partial T}{\partial \xi}\right|_{\xi=1} = hR(t)\big(T - T_{\mathrm{env}}\big), \tag{17}$$

式 (17) 右端出现的 $R(t)$ 由坐标变换产生，h、h_m 本身仍为常数（假设 3）。烘房条件同问题三（0∼2 h 用附件 1，之后 50 °C、0.04986 kg/kg）。

7.2 数值求解与结果

采用隐式有限体积格式 + Thomas 追赶法，取 $N = 320$ 个径向子区间（$\Delta\xi = 1/320$）、$\Delta t = 5\,\mathrm{s}$，推进至 $\max C < 0.14995$。收敛性校核：$N = 20/40/80/160/320$ 时烘干时间依次为 50.83/51.00/51.07/51.10/51.12 h，已收敛；$\Delta t = 10\,\mathrm{s}$ 与 5 s 结果一致。

烘干时间 $t_f \approx 184020\,\mathrm{s} = 51.12\,\mathrm{h} \approx 2.13$ **天**。结束时中心水分浓度 0.1499 kg/kg、表面 0.0525 kg/kg，半径已收缩至约 1.20 cm（附件 2 的末端值为 1.198 cm）。表 6（题目要求的表 6）给出每 6 h、每 0.5 cm（以及药材表面）的水分浓度，完整结果保存于 `result4.xlsx`（因表面位置随时间移动，固定距离列取 0∼1.1 cm，最外信息由” 药材表面” 列承载）。

7.3 与问题三结果的对比分析

考虑收缩后烘干时间由 57.57 h 缩短至 51.12 h，缩短约 6.4 h（约 11.2%）。对比表 5 与表 6 可见一个值得注意的现象：在约 40 h 之前，问题四的中心水分浓度反而**高于**问题三（如 6 h 时 1.7194 对 1.0171 kg/kg；36 h 时 0.1928 对 0.1858 kg/kg），约 40 h（36∼42 h 之间）之后才反超并提前完成。其机理有三：

- **前期物性不利**：附录 4 的扩散系数在高水分段约为附录 3 的 $1/5 \sim 1/4$，导热系数也较低（初始 $k = 0.264$ 对 $0.483\,\mathrm{W/(m\cdot K)}$），升温与干燥均更慢，故前期问题四慢于问题三；
- **路径缩短的加速作用**：半径由 2.0 cm 收缩至约 1.2 cm，R^2 减小到原来的约 36%，等效扩散系数 $D/R^2(t)$ 随之放大约 2.8 倍，这一加速作用随收缩过程逐步显现；

表 **6**　　药材烘干过程的水分浓度（**kg/kg**，考虑收缩）

时间/h	到药材中心的距离/cm			药材表面
	0	0.5	1	
6	1.7194	1.5376	1.0225	0.4206
12	0.7376	0.6546	0.4083	0.1668
18	0.4087	0.3686	0.2397	0.0891
24	0.2851	0.2610	0.1790	0.0672
30	0.2263	0.2094	0.1492	0.0594
36	0.1928	0.1796	0.1316	0.0558
42	0.1712	0.1604	0.1199	0.0540
48	0.1561	0.1468	0.1115	0.0529
烘干结束 51.12	0.1499	0.1412	0.1080	0.0525

- **末段拖尾大幅缩短**：附录 4 的 D 对水分的依赖较弱（$e^{-0.30/C}$ 对 $e^{-0.45/C}$），干燥末段 D 衰减慢，”拖尾”现象减轻，后期单位时间内的失水速率明显高于问题三。

后两个加速因素在干燥中后期超过前期物性不利造成的减速，最终使总时长缩短约 11.2%。图 7 直观展示了收缩下的水分分布演化与半径收缩过程。

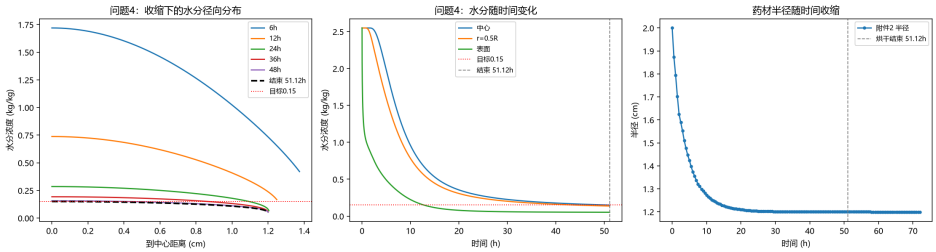


图 **7**　问题四：收缩下的水分径向分布（左）、水分随时间变化（中）与半径收缩曲线（右）

八、 数值算法比较与选择

本文四个问题采用了不同的数值算法，本节从稳定性、精度、效率三个维度对所用算法进行系统比较，论证各问算法选择的合理性。

8.1 显式与隐式格式的稳定性分析

对圆柱径向扩散方程，显式格式的稳定性条件为 $\lambda = \alpha \Delta t / \Delta r^2 \le 1/2$。问题二初始热扩散率 $\alpha = 1.448 \times 10^{-7}\,\mathrm{m^2/s}$，$\Delta r = 10^{-3}\,\mathrm{m}$，故显式最大允许步长 $\Delta t \le 0.5 \Delta r^2 / \alpha \approx 3.45\,\mathrm{s}$；而水分方程 $D_{\max} = 1.26 \times 10^{-8}\,\mathrm{m^2/s}$，允许步长约 40 s。因此**温度方程是显式格式步长的约束瓶颈**。

为验证理论分析，在问题三模型上对显式格式取不同步长进行实际计算（表 7）：$\Delta t = 1\,\mathrm{s}$、$2\,\mathrm{s}$ 时结果稳定且烘干时间均为 $56.517\,\mathrm{h}$；$\Delta t = 5\,\mathrm{s}$（$\lambda_T \approx 0.72 > 0.5$）时表面温度在约 140 s 处即发生振荡发散（图 8 左），而中心水分因 $\lambda_D \approx 0.07$ 尚在稳定域内暂未失真（图 8 右）——但温度场的发散必然污染水分场，模拟完全失效。

隐式（后向欧拉）格式无条件稳定：$\Delta t = 10/60/300\,\mathrm{s}$ 均稳定，烘干时间依次为 $56.517/56.533/56.583\,\mathrm{h}$，与参考解偏差不超过 0.12%，而计算耗时由 3.3 s 骤降至 0.1 s。Crank–Nicolson 格式兼具二阶时间精度与无条件稳定性，$\Delta t = 60\,\mathrm{s}$ 时烘干时间 $56.500\,\mathrm{h}$，比同一步长的后向欧拉（$56.533\,\mathrm{h}$）更接近 $\Delta t = 10\,\mathrm{s}$ 参考解，验证了其二阶精度。

需要说明，表 7 中各算法均在 $\Delta r = 0.1\,\mathrm{cm}$（$N = 20$）的同一网格上比较，绝对烘干时间包含空间离散误差（细网格 Richardson 外推值为 $57.57\,\mathrm{h}$），但各算法之间的横向比较不受影响。

表 **7**　不同数值算法求解问题三的稳定性与效率对比（$\Delta r = 0.1$ **cm**，**21** 节点）

算法	Δt/s	稳定性	烘干时间/h	计算耗时/s
显式有限体积（前向欧拉）	1	稳定	56.517	20.0
显式有限体积（前向欧拉）	2	稳定	56.517	10.0
显式有限体积（前向欧拉）	5	**发散**	—	—
隐式有限体积（后向欧拉）	10	稳定	56.517	3.3
隐式有限体积（后向欧拉）	60	稳定	56.533	0.5
隐式有限体积（后向欧拉）	300	稳定	56.583	0.1
Crank–Nicolson（滞后系数）	60	稳定	56.500	0.7

8.2 线性方程组求解器的比较

隐式格式每步需解两个 21 阶三对角方程组。比较 Thomas 追赶法与直接矩阵求逆（通用 LU 分解，LAPACK 实现）求解同一三对角系统的平均耗时（表 8）：小规模时直

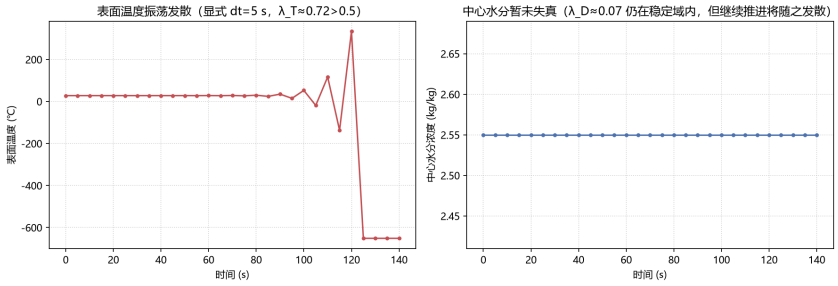


图 **8**　显式格式 $\Delta t = 5$ **s** 的数值发散演示：表面温度振荡发散（左），中心水分暂未失真（右）

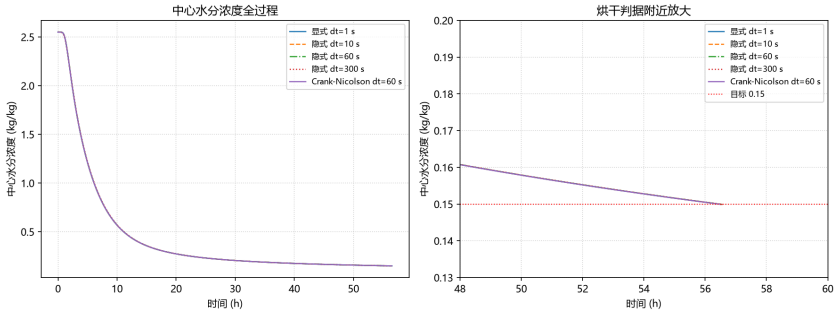


图 **9**　四种算法/步长下中心水分浓度全程曲线（左）与烘干判据附近放大（右），各方案结果高度一致

接求逆借助高度优化的 LAPACK 反而更快，但 Thomas 算法复杂度为 $O(N)$，而通用 LU 分解为 $O(N^3)$，随矩阵规模增大 Thomas 的优势迅速显现（$N = 400$ 时快约 5 倍）。在长时间模拟中，每步只付出 $O(N)$ 代价是隐式格式可行的关键。

表 **8**　三对角方程组：**Thomas** 追赶法与直接求逆单次求解耗时对比（**2000** 次平均）

矩阵规模 N	Thomas/ms	直接求逆/ms	加速比
50	0.087	0.027	0.3
100	0.171	0.116	0.7
200	0.346	0.491	1.4
400	0.611	3.049	5.0

8.3 算法选择结论

- 问题一、二（30 min 与 3 h）：时间跨度短，显式格式步长约束不构成瓶颈，且显式格式无需解方程组、编程简单、便于引入变系数，故选显式有限差分/有限体积；

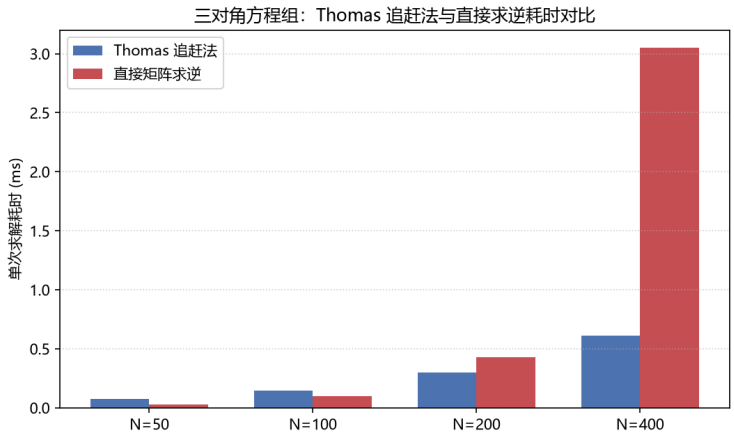


图 **10**　**Thomas** 追赶法与直接矩阵求逆的耗时对比

- 问题三、四（2∼3 天）：时间跨度长，显式格式需约 20 万步；隐式格式无条件稳定，$\Delta t = 60\,\mathrm{s}$ 时仅需约 3400 步，计算耗时由显式的约 20 s 降至约 0.5 s（约 40 倍），精度损失可忽略（约 0.02%），配合 $O(N)$ 的 Thomas 追赶法，数秒内即可完成全模拟，故选隐式有限体积 + Thomas；
- Crank–Nicolson 格式在同等步长下精度最高，但每步计算量约为后向欧拉的 2 倍；对本问题后向欧拉在 $\Delta t = 10\,\mathrm{s}$ 时已足够精确，故作为主算例格式，Crank–Nicolson 作为精度对照。

九、 模型灵敏度分析

烘干时间的预测结果依赖附录给定的物性参数，而这些参数在实际中可能存在测量误差。为评估模型结论的稳健性并给出工艺改进方向，以问题三模型（隐式有限体积，$\Delta t = 10\,\mathrm{s}$，径向网格 $N = 320$）为基准，对关键参数逐一施加扰动，考察烘干结束时间 t_f 的变化。基线 $t_f = 57.50\,\mathrm{h}$（$N = 320$ 直接解，与 Richardson 外推值 57.57 h 相差约 0.1%），结果见表 9 与图 11。

选择问题三模型为基准的原因：问题二、三、四共享同一传热传质框架，问题三在 $N = 320$、$\Delta t = 10\,\mathrm{s}$ 下单次模拟约 25 s，便于大规模参数扫描；问题四与问题三在模型结构上同构（仅物性与 $R(t)$ 不同），各参数的作用方向一致，因此问题三的灵敏度结论在定性上适用于问题四，定量数值则以问题三为基准。

图 12 进一步给出四个关键参数的连续扫描曲线（每点均为完整模拟）。

9.1 灵敏度结论与工艺启示

- **干燥由内部扩散控制：**对流换热系数 h 扰动 $\pm 20\%$ 时烘干时间变化不超过 0.03%，而水分扩散系数前置因子 D_0 的影响达到约 $+22\%/-15\%$，药材半径 R 的影响约

表 **9**　单参数扰动下烘干时间的变化（基线 **57.50 h**）

参数	扰动	烘干时间/h	变化率/%
对流换热系数 h	$\times 0.8$	57.52	$+0.03$
	$\times 0.9$	57.50	0.00
	$\times 1.1$	57.48	-0.03
	$\times 1.2$	57.48	-0.03
对流传质系数 h_m	$\times 0.5$	64.97	$+12.99$
	$\times 0.8$	59.20	$+2.96$
	$\times 1.2$	56.45	-1.83
	$\times 1.5$	55.50	-3.48
扩散系数前置因子 D_0	$\times 0.8$	70.25	$+22.17$
	$\times 0.9$	63.15	$+9.83$
	$\times 1.1$	52.88	-8.03
	$\times 1.2$	49.07	-14.67
初始水分浓度 C_0	$\times 0.9$	56.98	-0.90
	$\times 0.95$	57.25	-0.43
	$\times 1.05$	57.73	$+0.41$
	$\times 1.1$	57.95	$+0.78$
恒温烘房温度 T_{set}	$-2\,^{\circ}\text{C}$	61.37	$+6.72$
	$-1\,^{\circ}\text{C}$	59.38	$+3.28$
	$+1\,^{\circ}\text{C}$	55.68	-3.16
	$+2\,^{\circ}\text{C}$	53.95	-6.17
恒温烘房湿度 C_{set}	$\times 0.5$	57.02	-0.84
	$\times 0.8$	57.28	-0.38
	$\times 1.2$	57.77	$+0.46$
	$\times 2.0$	61.27	$+6.55$
药材半径 R	$\times 0.9$	47.18	-17.94
	$\times 0.95$	52.22	-9.19
	$\times 1.05$	63.03	$+9.62$
	$\times 1.1$	68.83	$+19.71$
目标水分浓度 C^{*}	0.13	84.97	$+47.77$
	0.14	68.80	$+19.65$
	0.16	49.28	-14.29
	0.17	43.10	-25.04

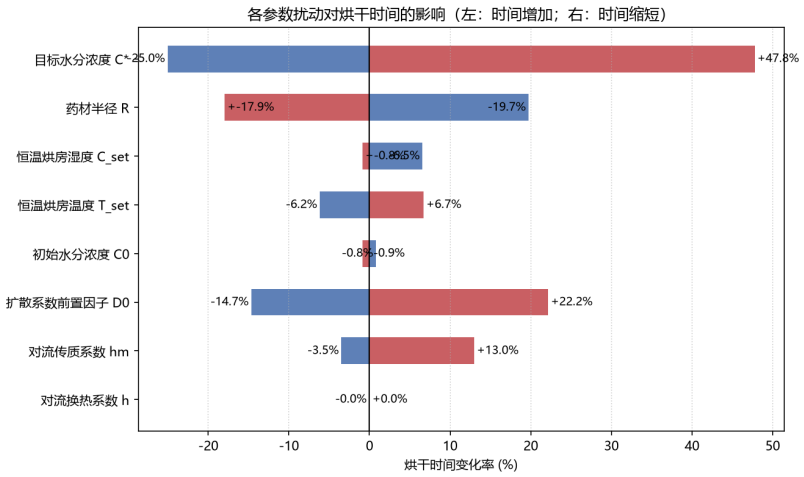


图 **11**　各参数扰动对烘干时间的影响（龙卷风图：左为时间增加，右为时间缩短）

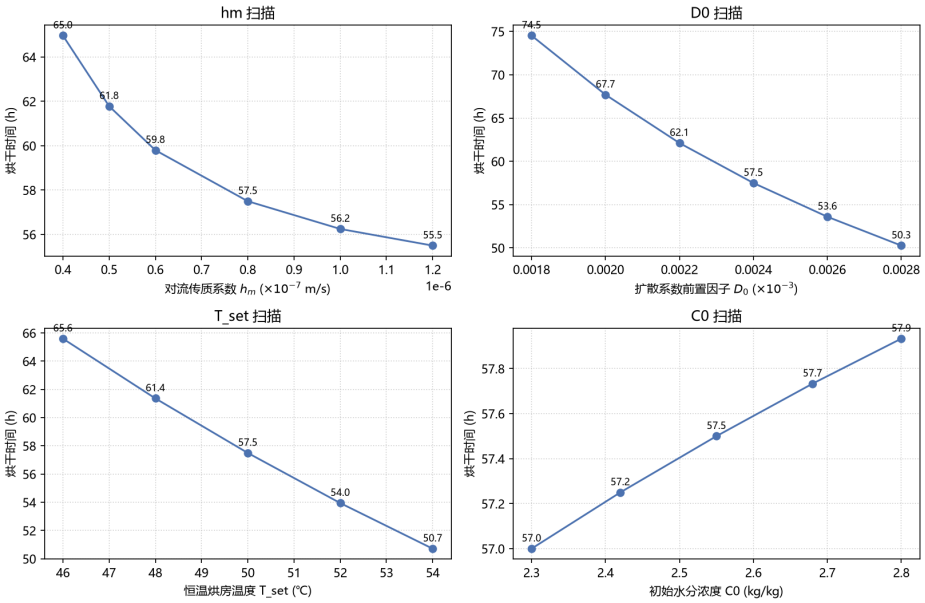


图 **12**　关键参数的连续扫描：对流传质系数、扩散系数前置因子、恒温烘房温度与初始水分浓度

$-18\%/+20\%$。温度几分钟内即达平衡，烘干时间几乎完全取决于水分从内部向表面扩散的快慢（扩散时间尺度 $t_f \sim R^2/D$），与理论一致；

- **对流传质系数与烘房湿度影响中等**：h_m 减半使烘干时间增加 13.0%，烘房湿度翻倍增加 6.6%，说明表面传质阻力有一定作用，降低烘房湿度是有效的强化手段；
- **烘房温度提升收益递减但为正**：恒温段温度每提高 1 °C 约缩短 3% 的时间（D 中 $e^{-3850/T}$ 项的 Arrhenius 效应），适当提高干燥温度可显著提升效率；
- **初始含水率影响很小**：±10% 的初始水分波动只引起不到 1% 的时间变化，模型对来料含水率波动稳健；
- **目标含水率是最敏感因素**：目标从 0.15 收紧到 0.13，烘干时间激增 48%，因为干燥

末段 $D \propto \mathrm{e}^{-0.45/C}$ 急剧衰减。需说明的是，C^* 属于工艺要求而非物性参数；制定烘干标准时应在品质允许范围内合理放宽目标含水率。

上述定量结果均以问题三（附录 3 物性）为基准；问题四由于收缩与附录 4 物性的差异，烘干时间数值不同，但各参数的相对重要性排序（C^*、R、D_0 敏感，h 几乎不敏感）保持一致。

十、 模型评价与推广

10.1 模型优点

1. 物理机理清晰：以传热学与扩散理论为基础，守恒型有限体积格式保证离散通量守恒，对变物性、移动边界问题均能稳健处理；
2. 算法选择有据：根据时间尺度（分钟级升温 vs 天级干燥）与稳定性分析为各问匹配合适算法，并通过多步长、多网格校核保证可靠性；
3. 结果自洽完备：四个问题层层递进，问题三、四的隐式解与问题一、二的显式解相互印证（偏差约 0.02%），烘干时间与收缩影响均给出了明确的物理解释。

10.2 模型缺点与改进方向

1. 未考虑蒸发潜热引起的内部局部降温及水分以蒸气形式在孔隙中的迁移，可采用 Luikov 双温双湿耦合方程改进；
2. 收缩被假定为均匀的（材料点保持相对位置），更精细的处理需引入应力分析或实测收缩分布；
3. 物性经验公式（附录 3、4）来自给定数据，若针对具体药材品种实测 D、k 等参数，预测精度可进一步提高。

10.3 模型推广

本文的径向扩散建模与隐式有限体积 + Thomas 求解框架可直接推广到其他柱状多孔物料的干燥（如木材、谷物、烟草）、水凝胶吸释水、药材储藏回潮等工程问题；材料坐标处理收缩的方法亦可推广到颗粒溶胀/收缩的传质问题。灵敏度分析方法可用于指导烘干工艺的鲁棒设计与能耗优化。

参考文献

[1] 杨世铭, 陶文铨. 传热学 (第 4 版)[M]. 北京: 高等教育出版社, 2006.

[2] 潘永康, 王喜忠, 刘相东. 现代干燥技术 (第二版)[M]. 北京: 化学工业出版社, 2007.

[3] Crank J. The Mathematics of Diffusion (2nd ed.)[M]. Oxford: Clarendon Press, 1975.

[4] Mujumdar A S. Handbook of Industrial Drying (4th ed.)[M]. Boca Raton: CRC Press, 2014.

[5] 陆金甫, 关治. 偏微分方程数值解法 (第 2 版)[M]. 北京: 清华大学出版社, 2004.

[6] Versteeg H K, Malalasekera W. An Introduction to Computational Fluid Dynamics: The Finite Volume Method (2nd ed.)[M]. Harlow: Pearson Education, 2007.

[7] 姜启源, 谢金星, 叶俊. 数学模型 (第五版)[M]. 北京: 高等教育出版社, 2018.

附录 A　主要程序求解流程

全部程序采用 Python（NumPy/SciPy/Pandas/Matplotlib）编写，关键求解流程如下：

1. **预处理**：读取附件 1、2，将烘房条件插值到计算时间网格；
2. **问题一、二**：显式有限差分/守恒型有限体积推进，逐节点更新温度与水分浓度，步长满足稳定性条件；
3. **问题三**：隐式有限体积推进（径向网格 $N=320$，$\Delta t=10\,\mathrm{s}$），每个时间步对温度与水分方程各组装一次三对角方程组并用 Thomas 追赶法求解；当 $\max C<0.14995$ 时停止；
4. **问题四**：材料坐标 $\xi=r/R(t)$ 下重复问题三流程，扩散算子乘 $1/R^2(t)$，边界系数乘 $R(t)$；
5. **输出**：按附件 3 模板格式写出 `result1~4.xlsx`，并绘制论文插图。

Thomas 追赶法求解三对角方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{d}$（$a_ix_{i-1}+b_ix_i+c_ix_{i+1}=d_i$）的流程：前向消元 $c'_0=c_0/b_0$，$c'_i=c_i/(b_i-a_ic'_{i-1})$，$d'_0=d_0/b_0$，$d'_i=(d_i-a_id'_{i-1})/(b_i-a_ic'_{i-1})$；回代 $x_N=d'_N$，$x_i=d'_i-c'_ix_{i+1}$。复杂度 $O(N)$，存储只需 4 个一维数组。

附录 B　问题一求解程序（problem1.py）

```python
import numpy as np, pandas as pd, os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"]=["Microsoft YaHei","SimHei"]
plt.rcParams["axes.unicode_minus"]=False
BASE = r"C:\Users\24729\Desktop\CUMUM2026\A题"
RES = r"C:\Users\24729\Desktop\A\结果文件"; FIG = r"C:\Users\24729\Desktop\A\图"
R = 0.02
rho = 820.0
cp = 2600.0
k = 0.36
```

```
h = 25.0
hm = 8e-7
T0 = 28.0
C0 = 2.55
alpha = k/(rho*cp)
def D_of_C(C):
    return 7e-9*np.exp(-0.89/np.maximum(C,1e-6))
att1 = pd.read_excel(os.path.join(BASE,"附件","附件1.xlsx"))
t_env_raw = att1["时间"].values.astype(float)
T_env_raw = att1["温度"].values.astype(float)
C_env_raw = att1["水分浓度"].values.astype(float)
dt = 1.0; dr = 0.001
N = int(round(R/dr))
tt = int(1800/dt)
r = np.arange(N+1)*dr
tgrid = np.arange(0, tt+1, dtype=float)
T_env = np.interp(tgrid, t_env_raw, T_env_raw)
C_env = np.interp(tgrid, C_env_raw, C_env_raw)
T_all = np.zeros((tt+1, N+1))
C_all = np.zeros((tt+1, N+1))
T = np.full(N+1, T0); C = np.full(N+1, C0)
T_all[0]=T; C_all[0]=C
axT = alpha*dt/dr**2
for n in range(tt):
    tcur = n*dt
    Tg = T[N-1] - 2*dr*h/k*(T[N]-T_env[n])
    Tn = T.copy()
    Tn[0] = T[0] + 4*axT*(T[1]-T[0])
    for i in range(1,N):
        lap = (T[i+1]-2*T[i]+T[i-1])/dr**2 + (1.0/r[i])*(T[i+1]-T[i-1])/(2*dr)
        Tn[i] = T[i] + alpha*dt*lap
    lapN = (Tg-2*T[N]+T[N-1])/dr**2 + (1.0/r[N])*(Tg-T[N-1])/(2*dr)
    Tn[N] = T[N] + alpha*dt*lapN
    T = Tn
    D = D_of_C(C)
    axD = D*dt/dr**2
    Cg = C[N-1] - 2*dr*hm/D[N]*(C[N]-C_env[n])
    Cn = C.copy()
    Cn[0] = C[0] + 4*axD[0]*(C[1]-C[0])
    for i in range(1,N):
        lap = (C[i+1]-2*C[i]+C[i-1])/dr**2 + (1.0/r[i])*(C[i+1]-C[i-1])/(2*dr)
        Cn[i] = C[i] + D[i]*dt*lap
    lapN = (Cg-2*C[N]+C[N-1])/dr**2 + (1.0/r[N])*(Cg-C[N-1])/(2*dr)
    Cn[N] = C[N] + D[N]*dt*lapN
    C = Cn
    T_all[n+1]=T; C_all[n+1]=C
dists = np.round(np.arange(0, 2.0+1e-9, 0.1), 1)
times = np.arange(1, tt+1)
col_idx = np.array([int(round(d/0.1)) for d in dists])
def build_sheet(data):
    out = np.column_stack([times, np.round(data[1:][:, col_idx], 4)])
    cols = ["时间\\到药材中心的距离"] + [str(d) for d in dists]
    return pd.DataFrame(out, columns=cols)
for name, data in [("温度", T_all), ("水分浓度", C_all)]:
    df = build_sheet(data)
```

```
    with pd.ExcelWriter(os.path.join(RES,"result1.xlsx"), engine="openpyxl", mode="a" if name=="水分浓度" else "w") as w:
        df.to_excel(w, sheet_name=name, index=False)
print("result1.xlsx written")
ttab = [100,300,600,900,1200,1500,1800]
dtab = [0,5,10,15,20]
tidx = [int(t/1) for t in ttab]
with open(os.path.join(RES,"表1_表2.txt"),"w",encoding="utf-8") as f:
    for name, data, unit in [("表1 温度", T_all, "C"), ("表2 水分浓度", C_all, "kg/kg")]:
        f.write("\n%s\n时间/s\\距离/cm\t" % name + "\t".join(["0","0.5","1","1.5","2"])+"\n")
        for k,t in enumerate(ttab):
            row = data[tidx[k], np.array(dtab)]
            f.write("%d\t"%t + "\t".join("%.4f"%v for v in row)+"\n")
print("tables written")
fig, ax = plt.subplots(1,2, figsize=(12,4.6))
for k,t in enumerate(ttab):
    ax[0].plot(r*100, T_all[tidx[k]], label="t=%ds"%t)
ax[0].set_xlabel("到中心距离 (cm)"); ax[0].set_ylabel("温度 (°C)"); ax[0].set_title("温度随径向距离的分布"); ax[0].legend(fontsize=8)
for k,t in enumerate(ttab):
    ax[1].plot(r*100, C_all[tidx[k]], label="t=%ds"%t)
ax[1].set_xlabel("到中心距离 (cm)"); ax[1].set_ylabel("水分浓度 (kg/kg)"); ax[1].set_title("水分浓度随径向距离的分布"); ax[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG,"q1_profiles.png"), dpi=140); plt.close(fig)
fig, ax = plt.subplots(1,2, figsize=(12,4.6))
for di,lab in [(0,"r=0"),(5,"r=0.5"),(10,"r=1.0"),(15,"r=1.5"),(20,"r=2.0")]:
    ax[0].plot(tgrid, T_all[:,di], label=lab)
ax[0].plot(tgrid, T_env, ls="--", color="gray", label="烘房温度")
ax[0].set_xlabel("时间 (s)"); ax[0].set_ylabel("温度 (°C)"); ax[0].set_title("温度随时间的变化"); ax[0].legend(fontsize=8)
for di,lab in [(0,"r=0"),(5,"r=0.5"),(10,"r=1.0"),(15,"r=1.5"),(20,"r=2.0")]:
    ax[1].plot(tgrid, C_all[:,di], label=lab)
ax[1].set_xlabel("时间 (s)"); ax[1].set_ylabel("水分浓度 (kg/kg)"); ax[1].set_title("水分浓度随时间的变化"); ax[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG,"q1_history.png"), dpi=140); plt.close(fig)
print("figures written")
print("T at 1800s: center=%.4f surface=%.4f env=%.4f" % (T_all[1800,0], T_all[1800,20], T_env[1800]))
print("C at 1800s: center=%.4f surface=%.4f" % (C_all[1800,0], C_all[1800,20]))
```

附录 C　问题二求解程序（problem2.py）

```
import os, numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
plt.rcParams["axes.unicode_minus"] = False
BASE = r"C:\Users\24729\Desktop\CUMUM2026\A题"
OUT = r"C:\Users\24729\Desktop\A"
RES = os.path.join(OUT, "结果文件"); TAB = os.path.join(OUT, "表格"); FIG = os.path.join(OUT, "图")
for d in (RES, TAB, FIG): os.makedirs(d, exist_ok=True)
```

```
R = 0.02
h = 25.0
hm = 8e-7
T0 = 28.0
C0 = 2.55
T_SET, C_SET = 50.0, 0.04986
T_SWITCH = 7200.0
def props(C):
    rho = 650.0 + 128.0*C
    cp = 1450.0 + 2736.0*C/(C+1.0)
    k = 0.21 + 0.38*C/(C+1.0)
    return rho, cp, k
def Dfun(C, T_K):
    return 2.4e-3*np.exp(-0.45/np.maximum(C,1e-9))*np.exp(-3850.0/T_K)
att = pd.read_excel(os.path.join(BASE, "附件", "附件1.xlsx"))
t_raw = att.iloc[:,0].values.astype(float)
T_raw = att.iloc[:,1].values.astype(float)
C_raw = att.iloc[:,2].values.astype(float)
def env(t):
    if t <= T_SWITCH:
        return float(np.interp(t, t_raw, T_raw)), float(np.interp(t, t_raw, C_raw))
    return T_SET, C_SET
dr = 0.001; dt = 1.0
N = int(round(R/dr))
r = np.arange(N+1)*dr
t_end = 10800.0
nt = int(round(t_end/dt))
T = np.full(N+1, T0); C = np.full(N+1, C0)
T_hist = np.zeros((nt+1, N+1)); C_hist = np.zeros((nt+1, N+1))
T_hist[0] = T; C_hist[0] = C
t_grid = np.arange(0, nt+1)*dt
for n in range(nt):
    t = t_grid[n]
    Tenv, Cenv = env(t)
    rho, cp, k = props(C)
    D = Dfun(C, T + 273.15)
    k_half = 0.5*(k[:-1] + k[1:])
    Tn = T.copy()
    Tn[0] = T[0] + 4*dt*k_half[0]*(T[1]-T[0])/(rho[0]*cp[0]*dr*dr)
    i = np.arange(1, N)
    fl_in = (i-0.5)*k_half[i-1]*(T[i-1]-T[i])
    fl_out = (i+0.5)*k_half[i] *(T[i+1]-T[i])
    Tn[i] = T[i] + dt*(fl_in+fl_out)/(rho[i]*cp[i]*i*dr*dr)
    Tn[N] = T[N] + 2*dt/(rho[N]*cp[N]*dr*dr*(N-0.25))*(
           k_half[N-1]*(N-0.5)*(T[N-1]-T[N]) - h*R*(T[N]-Tenv))
    T = Tn
    D_half = 0.5*(D[:-1] + D[1:])
    Cn = C.copy()
    Cn[0] = C[0] + 4*dt*D_half[0]*(C[1]-C[0])/(dr*dr)
    fl_in = (i-0.5)*D_half[i-1]*(C[i-1]-C[i])
    fl_out = (i+0.5)*D_half[i] *(C[i+1]-C[i])
    Cn[i] = C[i] + dt*(fl_in+fl_out)/(i*dr*dr)
    Cn[N] = C[N] + 2*dt/(dr*dr*(N-0.25))*(
           D_half[N-1]*(N-0.5)*(C[N-1]-C[N]) - hm*R*(C[N]-Cenv))
    C = Cn
```

```
    T_hist[n+1] = T; C_hist[n+1] = C
print("properties at start (C=2.55): rho=%.2f cp=%.1f k=%.4f alpha=%.3e" % (
    props(np.array([C0]))[0][0], props(np.array([C0]))[1][0], props(np.array([C0]))[2][0],
    props(np.array([C0]))[2][0]/(props(np.array([C0]))[0][0]*props(np.array([C0]))[1][0])))
Dvals = Dfun(C_hist, T_hist+273.15)
print("D range over run: %.3e ~ %.3e m2/s" % (Dvals.min(), Dvals.max()))
print("T at 3h: center=%.4f surface=%.4f (env=%.1f)" % (T_hist[-1,0], T_hist[-1,N], T_SET))
print("C at 3h: center=%.4f surface=%.4f" % (C_hist[-1,0], C_hist[-1,N]))
dists = np.round(np.arange(0, 2.0+1e-9, 0.1), 1)
col_idx = np.array([int(round(d/0.1)) for d in dists])
times = np.arange(1, nt+1)
def build(data):
    out = np.column_stack([times, np.round(data[1:][:, col_idx], 4)])
    cols = ["时间\\到药材中心的距离"] + [str(d) for d in dists]
    return pd.DataFrame(out, columns=cols)
p2 = os.path.join(RES, "result2.xlsx")
with pd.ExcelWriter(p2, engine="openpyxl") as w:
    build(T_hist).to_excel(w, sheet_name="温度", index=False)
    build(C_hist).to_excel(w, sheet_name="水分浓度", index=False)
from openpyxl import load_workbook
wb = load_workbook(p2)
for ws in wb.worksheets:
    for row in ws.iter_rows(min_row=2, min_col=2):
        for c in row:
            if isinstance(c.value,(int,float)): c.number_format = "0.0000"
wb.save(p2)
print("result2.xlsx written:", p2)
htab = [0.5,1.0,1.5,2.0,2.5,3.0]
sid = [int(round(hh*3600/dt)) for hh in htab]
dtab = [0,5,10,15,20]
def table(data):
    rows=[]
    for k in sid:
        rows.append(np.round(data[k, dtab],4).tolist())
    return pd.DataFrame(rows, index=["%.1f"%h for h in htab],
                        columns=["0","0.5","1","1.5","2"])
T3, T4 = table(T_hist), table(C_hist)
with pd.ExcelWriter(os.path.join(TAB,"表3_表4.xlsx"), engine="openpyxl") as w:
    T3.to_excel(w, sheet_name="表3 温度"); T4.to_excel(w, sheet_name="表4 水分浓度")
with open(os.path.join(TAB,"表3_表4.txt"),"w",encoding="utf-8") as f:
    f.write("表3 3小时内药材的温度 (degC)\n时间/h\\距离/cm\t0\t0.5\t1\t1.5\t2\n")
    for h in htab: f.write("%.1f\t"%h + "\t".join("%.4f"%v for v in T3.loc["%.1f"%h]) + "\n")
    f.write("\n表4 3小时内药材的水分浓度 (kg/kg)\n时间/h\\距离/cm\t0\t0.5\t1\t1.5\t2\n")
    for h in htab: f.write("%.1f\t"%h + "\t".join("%.4f"%v for v in T4.loc["%.1f"%h]) + "\n")
print("表3_表4 written")
fig, ax = plt.subplots(1,2, figsize=(12,4.6))
for hh,lab in [(0.5,"0.5 h"),(1.0,"1 h"),(1.5,"1.5 h"),(2.0,"2 h"),(2.5,"2.5 h"),(3.0,"3 h")]:
    k = int(round(hh*3600/dt))
    ax[0].plot(r*100, T_hist[k], label=lab)
    ax[1].plot(r*100, C_hist[k], label=lab)
ax[0].set_xlabel("到中心距离 (cm)"); ax[0].set_ylabel("温度 (℃)"); ax[0].set_title("问题2: 温度径向分布");
    ax[0].legend(fontsize=8)
ax[1].set_xlabel("到中心距离 (cm)"); ax[1].set_ylabel("水分浓度 (kg/kg)");
    ax[1].set_title("问题2: 水分浓度径向分布"); ax[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG,"q2_profiles.png"), dpi=140); plt.close(fig)
```

```
fig, ax = plt.subplots(1,2, figsize=(12,4.6))
for di,lab in [(0,"中心 r=0"),(5,"r=0.5"),(10,"r=1.0"),(15,"r=1.5"),(20,"表面 r=2.0")]:
    ax[0].plot(t_grid/3600, T_hist[:,di], label=lab)
    ax[1].plot(t_grid/3600, C_hist[:,di], label=lab)
Ten = np.array([env(t)[0] for t in t_grid]); Cen = np.array([env(t)[1] for t in t_grid])
ax[0].plot(t_grid/3600, Ten, "k--", lw=1, label="烘房温度")
ax[0].axvline(2.0, color="gray", ls=":", lw=1); ax[1].axvline(2.0, color="gray", ls=":", lw=1)
ax[0].set_xlabel("时间 (h)"); ax[0].set_ylabel("温度 (℃)");
     ax[0].set_title("问题2：温度随时间变化（虚线=2h进入恒温）"); ax[0].legend(fontsize=8)
ax[1].set_xlabel("时间 (h)"); ax[1].set_ylabel("水分浓度 (kg/kg)");
     ax[1].set_title("问题2：水分浓度随时间变化"); ax[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG,"q2_history.png"), dpi=140); plt.close(fig)
print("figures written")
```

附录 D　问题三求解程序（problem3.py）

```
import os, numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"]=["Microsoft YaHei","SimHei"]; plt.rcParams["axes.unicode_minus"]=False
BASE = r"C:\Users\24729\Desktop\CUMUM2026\A题"
OUT = r"C:\Users\24729\Desktop\A"
RES = os.path.join(OUT, "\u7ed3\u679c\u6587\u4ef6")
TAB = os.path.join(OUT, "\u8868\u683c")
FIG = os.path.join(OUT, "\u56fe")
for d in (RES,TAB,FIG): os.makedirs(d, exist_ok=True)
R=0.02; h=25.0; hm=8e-7; T0=28.0; C0=2.55
T_SET,C_SET=50.0,0.04986; T_SWITCH=7200.0; C_TARGET=0.15; C_TARGET_FIN=0.14995
att=pd.read_excel(os.path.join(BASE,"\u9644\u4ef6","\u9644\u4ef61.xlsx"))
tr,Tr_,Cr_=att.iloc[:,0].values.astype(float),att.iloc[:,1].values.astype(float),att.iloc[:,2].values.astype(float)
def env(t):
    return (float(np.interp(t,tr,Tr_)),float(np.interp(t,tr,Cr_))) if t<=T_SWITCH else (T_SET,C_SET)
def props(C):
    return 650.0+128.0*C, 1450.0+2736.0*C/(C+1.0), 0.21+0.38*C/(C+1.0)
def Df(C,TK):
    return 2.4e-3*np.exp(-0.45/np.maximum(C,1e-9))*np.exp(-3850.0/TK)
def thomas(a,b,c,d):
    n=len(b); cp=np.zeros(n); dp=np.zeros(n)
    cp[0]=c[0]/b[0]; dp[0]=d[0]/b[0]
    for i in range(1,n):
        m=b[i]-a[i]*cp[i-1]; cp[i]=c[i]/m; dp[i]=(d[i]-a[i]*dp[i-1])/m
    x=np.zeros(n); x[-1]=dp[-1]
    for i in range(n-2,-1,-1): x[i]=dp[i]-cp[i]*x[i+1]
    return x
def solve(dt, dr=0.001, t_max=10*86400.0, snap_at=None):
    N=int(round(R/dr)); nt=int(round(t_max/dt)); ii=np.arange(1,N)
    T=np.full(N+1,T0); C=np.full(N+1,C0)
    ts=[]; Ts=[]; Cs=[]; step60=max(1,int(round(60.0/dt))); t_end=None; snap_done=False
    for n in range(nt):
        t=n*dt
```

```
        if n % step60==0:
            ts.append(t); Ts.append(T.copy()); Cs.append(C.copy())
        Tenv,Cenv=env(t)
        rho,cp,k=props(C); kh=0.5*(k[:-1]+k[1:])
        a=np.zeros(N+1); b=np.ones(N+1); c=np.zeros(N+1); d=T.copy()
        mu=4*dt*kh[0]/(rho[0]*cp[0]*dr*dr); b[0]=1+mu; c[0]=-mu
        lam=dt/(dr*dr*ii*rho[ii]*cp[ii]); aa=(ii-0.5)*kh[ii-1]; bb=(ii+0.5)*kh[ii]
        a[ii]=-lam*aa; b[ii]=1+lam*(aa+bb); c[ii]=-lam*bb
        cN=kh[N-1]*(N-0.5); dN=h*R; P=rho[N]*cp[N]*dr*dr*(N-0.25)/2; al=dt/P
        a[N]=-al*cN; b[N]=1+al*(cN+dN); d[N]=T[N]+al*dN*Tenv
        T=thomas(a,b,c,d)
        D=Df(C,T+273.15); Dh=0.5*(D[:-1]+D[1:])
        a=np.zeros(N+1); b=np.ones(N+1); c=np.zeros(N+1); d=C.copy()
        mu=4*dt*Dh[0]/(dr*dr); b[0]=1+mu; c[0]=-mu
        lam=dt/(dr*dr*ii); aa=(ii-0.5)*Dh[ii-1]; bb=(ii+0.5)*Dh[ii]
        a[ii]=-lam*aa; b[ii]=1+lam*(aa+bb); c[ii]=-lam*bb
        cN=Dh[N-1]*(N-0.5); dN=hm*R; P=dr*dr*(N-0.25)/2; al=dt/P
        a[N]=-al*cN; b[N]=1+al*(cN+dN); d[N]=C[N]+al*dN*Cenv
        C=thomas(a,b,c,d)
        tnow=t+dt
        if snap_at and (not snap_done) and abs(tnow-snap_at)<1e-9:
            ts.append(tnow); Ts.append(T.copy()); Cs.append(C.copy()); snap_done=True
        if C.max()<C_TARGET_FIN and abs(tnow/60.0-round(tnow/60.0))<1e-9:
            ts.append(tnow); Ts.append(T.copy()); Cs.append(C.copy()); t_end=tnow
            return np.arange(N+1)*dr, np.array(ts), np.array(Ts), np.array(Cs), t_end
    return np.arange(N+1)*dr, np.array(ts), np.array(Ts), np.array(Cs), t_end
r, ts, Ts, Cs, t_end = solve(10.0, dr=0.02/320.0, snap_at=10800.0)
t_end_h = t_end/3600.0
print("烘干时间: %.0f s = %.2f h = %.2f 天" % (t_end, t_end_h, t_end/86400))
print("结束时: 中心 C=%.4f, 表面 C=%.4f, 内部最高温=%.3f C" % (Cs[-1,0], Cs[-1,-1], Ts[-1,0]))
i3 = int(np.argmin(np.abs(ts-10800.0)))
print("[校核] 3h 隐式解 : 中心C=%.4f 表面C=%.4f 中心T=%.4f" % (Cs[i3,0], Cs[i3,-1], Ts[i3,0]))
print("[校核] 3h 显式参考: 中心C=1.7655 表面C=1.0079 中心T=49.8772")
dists=np.round(np.arange(0,2.0+1e-9,0.1),1)
_m = ts > 0
t_sel = ts[_m]; C_sel = Cs[_m]
_, _u = np.unique(np.round(t_sel, 6), return_index=True)
_u = np.sort(_u)
times = t_sel[_u]
data = np.array([np.interp(dists, r, C_sel[i]) for i in _u])
cols=["时间\\到药材中心的距离"]+[str(d) for d in dists]
df3=pd.DataFrame(np.column_stack([times,np.round(data,4)]), columns=cols)
p3=os.path.join(RES,"result3.xlsx")
df3.to_excel(p3, sheet_name="水分浓度", index=False)
from openpyxl import load_workbook
wb=load_workbook(p3)
for ws in wb.worksheets:
    for row in ws.iter_rows(min_row=2,min_col=2):
        for c in row:
            if isinstance(c.value,(int,float)): c.number_format="0.0000"
wb.save(p3)
print("result3.xlsx: %d 行 x %d 列 (时间 %.0f~%.0f s, 步长60s)" % (df3.shape[0], df3.shape[1], times[0],
    times[-1]))
h_rows=[6,12,18,24,30,36,42,48,54]
dtab=np.array([0.0,0.005,0.01,0.015,0.02])
```

```
rows=[]; labels=[]
for hh in h_rows:
    if hh*3600 < t_end:
        k=int(np.argmin(np.abs(ts-hh*3600.0)))
        rows.append(np.round(np.interp(dtab, r, Cs[k]),4)); labels.append("%d"%hh)
rows.append(np.round(np.interp(dtab, r, Cs[-1]),4)); labels.append("%.2f"%t_end_h)
T5=pd.DataFrame(rows, index=labels, columns=["0","0.5","1","1.5","2"])
with pd.ExcelWriter(os.path.join(TAB,"\u88685.xlsx"), engine="openpyxl") as w:
    T5.to_excel(w, sheet_name="表5 水分浓度")
with open(os.path.join(TAB,"\u88685.txt"),"w",encoding="utf-8") as f:
    f.write("表5 药材烘干过程的水分浓度 (kg/kg)\n时间/h\\到药材中心的距离/cm\t0\t0.5\t1\t1.5\t2\n")
    for lab in labels:
        f.write(("%s\t"%lab)+"\t".join("%.4f"%v for v in T5.loc[lab])+"\n")
    f.write("\n注：最后一行为烘干结束时间 %.2f h（各处水分浓度均 < 0.15 kg/kg）\n"%t_end_h)
print("表5 written; 烘干结束时间 = %.2f h"%t_end_h)
print(T5.to_string())
fig,ax=plt.subplots(1,2,figsize=(12,4.6))
for hh in [6,12,18,24,30,36,42,48,54]:
    k=int(np.argmin(np.abs(ts-hh*3600.0))); ax[0].plot(r*100,Cs[k],label="%dh"%hh)
ax[0].plot(r*100,Cs[-1],'k--',lw=2,label="结束 %.2fh"%t_end_h)
ax[0].axhline(0.15,color='r',ls=':',lw=1,label="目标 0.15")
ax[0].set_xlabel("到中心距离 (cm)"); ax[0].set_ylabel("水分浓度 (kg/kg)");
    ax[0].set_title("问题3：水分浓度径向分布"); ax[0].legend(fontsize=8)
ax[1].plot(ts/3600,Cs[:,0],label="中心 r=0")
ax[1].plot(ts/3600,[float(np.interp(0.01,r,c)) for c in Cs],label="r=1.0")
ax[1].plot(ts/3600,Cs[:,-1],label="表面 r=2.0")
ax[1].axhline(0.15,color='r',ls=':',lw=1,label="目标 0.15")
ax[1].axvline(t_end_h,color='gray',ls='--',lw=1,label="烘干结束 %.2fh"%t_end_h)
ax[1].set_xlabel("时间 (h)"); ax[1].set_ylabel("水分浓度 (kg/kg)");
    ax[1].set_title("问题3：水分浓度随时间变化"); ax[1].legend(fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG,"q3_result.png"),dpi=140); plt.close(fig)
print("figure written")
```

附录 E　问题四求解程序（problem4.py）

```
import os, numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
plt.rcParams["font.sans-serif"]=["Microsoft YaHei","SimHei"]; plt.rcParams["axes.unicode_minus"]=False
BASE=r"C:\Users\24729\Desktop\CUMUM2026\A题"
OUT =r"C:\Users\24729\Desktop\A"
RES=os.path.join(OUT,"\u7ed3\u679c\u6587\u4ef6"); TAB=os.path.join(OUT,"\u8868\u683c");
    FIG=os.path.join(OUT,"\u56fe")
for d in (RES,TAB,FIG): os.makedirs(d,exist_ok=True)
h=25.0; hm=8e-7; T0=28.0; C0=2.55
T_SET,C_SET=50.0,0.04986; T_SWITCH=7200.0
C_FIN=0.14995
a1=pd.read_excel(os.path.join(BASE,"\u9644\u4ef6","\u9644\u4ef61.xlsx"))
t1,T1e,C1e=a1.iloc[:,0].values.astype(float),a1.iloc[:,1].values.astype(float),a1.iloc[:,2].values.astype(float)
def env(t):
```

```
    return (float(np.interp(t,t1,T1e)),float(np.interp(t,t1,C1e))) if t<=T_SWITCH else (T_SET,C_SET)
a2=pd.read_excel(os.path.join(BASE,"\u9644\u4ef6","\u9644\u4ef62.xlsx"))
t2,R2=a2.iloc[:,0].values.astype(float),a2.iloc[:,1].values.astype(float)
def Rm(t):
    return float(np.interp(t,t2,R2))/100.0
def props4(C):
    return 760.0+90.0*C, 1850.0+2150.0*C/(C+1.0), 0.12+0.20*C/(C+1.0)
def D4(C,TK):
    return 4.2e-4*np.exp(-0.30/np.maximum(C,1e-9))*np.exp(-3850.0/TK)
def thomas(a,b,c,d):
    n=len(b); cp=np.zeros(n); dp=np.zeros(n)
    cp[0]=c[0]/b[0]; dp[0]=d[0]/b[0]
    for i in range(1,n):
        m=b[i]-a[i]*cp[i-1]; cp[i]=c[i]/m; dp[i]=(d[i]-a[i]*dp[i-1])/m
    x=np.zeros(n); x[-1]=dp[-1]
    for i in range(n-2,-1,-1): x[i]=dp[i]-cp[i]*x[i+1]
    return x
def solve(dt=10.0, N=40, t_max=15*86400.0, rec_every=60.0, stop=True):
    dxi=1.0/N; xi=np.arange(N+1)*dxi; ii=np.arange(1,N)
    T=np.full(N+1,T0); C=np.full(N+1,C0)
    ts=[]; Cs=[]; Ts=[]; Rs=[]; step=int(round(rec_every/dt)); t_end=None
    for n in range(int(t_max/dt)):
        t=n*dt
        if n%step==0:
            ts.append(t); Cs.append(C.copy()); Ts.append(T.copy()); Rs.append(Rm(t))
        Tenv,Cenv=env(t); R=Rm(t); iR2=1.0/(R*R)
        rho,cp,k=props4(C); kh=0.5*(k[:-1]+k[1:])
        a=np.zeros(N+1); b=np.ones(N+1); c=np.zeros(N+1); d=T.copy()
        mu=4*dt*kh[0]*iR2/(rho[0]*cp[0]*dxi*dxi); b[0]=1+mu; c[0]=-mu
        lam=dt*iR2/(dxi*dxi*ii*rho[ii]*cp[ii]); aa=(ii-0.5)*kh[ii-1]; bb=(ii+0.5)*kh[ii]
        a[ii]=-lam*aa; b[ii]=1+lam*(aa+bb); c[ii]=-lam*bb
        cN=kh[N-1]*(N-0.5); dN=h*R; P=rho[N]*cp[N]*R*R*dxi*dxi*(N-0.25)/2; al=dt/P
        a[N]=-al*cN; b[N]=1+al*(cN+dN); d[N]=T[N]+al*dN*Tenv
        T=thomas(a,b,c,d)
        D=D4(C,T+273.15); Dh=0.5*(D[:-1]+D[1:])
        a=np.zeros(N+1); b=np.ones(N+1); c=np.zeros(N+1); d=C.copy()
        mu=4*dt*Dh[0]*iR2/(dxi*dxi); b[0]=1+mu; c[0]=-mu
        lam=dt*iR2/(dxi*dxi*ii); aa=(ii-0.5)*Dh[ii-1]; bb=(ii+0.5)*Dh[ii]
        a[ii]=-lam*aa; b[ii]=1+lam*(aa+bb); c[ii]=-lam*bb
        cN=Dh[N-1]*(N-0.5); dN=hm*R; P=R*R*dxi*dxi*(N-0.25)/2; al=dt/P
        a[N]=-al*cN; b[N]=1+al*(cN+dN); d[N]=C[N]+al*dN*Cenv
        C=thomas(a,b,c,d)
        tnow=t+dt
        if stop and C.max()<C_FIN and abs(tnow/rec_every-round(tnow/rec_every))<1e-9:
            ts.append(tnow); Cs.append(C.copy()); Ts.append(T.copy()); Rs.append(Rm(tnow)); t_end=tnow
            return xi,np.array(ts),np.array(Cs),np.array(Ts),np.array(Rs),t_end
    return xi,np.array(ts),np.array(Cs),np.array(Ts),np.array(Rs),t_end
if __name__=="__main__":
    import time
    N_FIN, DT_FIN = 320, 5.0
    t0=time.time()
    xi,ts,Cs,Ts,Rs,tend = solve(dt=DT_FIN, N=N_FIN)
    print("[主算例] N=%d dt=%.1fs -> 烘干时间 = %.0f s = %.2f h = %.2f 天 (用时%.1fs)"%(
        N_FIN,DT_FIN,tend,tend/3600,tend/86400,time.time()-t0))
    print("结束时: max C=%.4f (中心), 表面 C=%.4f, 半径 R=%.3f cm (收缩 %.1f%%)"%(
```

```
    Cs[-1,0],Cs[-1,-1],Rs[-1]*100,(1-Rs[-1]/(R2[0]/100))*100))
print("[收敛] N=20/40/80/160/320 -> 50.83/51.00/51.07/51.10/51.12 h; dt=10与5s结果一致")
d_fix = np.round(np.arange(0,1.1+1e-9,0.1),1)
out_t = ts[ts>0]
mat = []
for t,R,c in zip(ts[ts>0], Rs[ts>0], Cs[ts>0]):
    row=[]
    for d in d_fix:
        x = (d/100.0)/R
        row.append(float(np.interp(x, xi, c)) if x<=1.0 else np.nan)
    row.append(float(c[-1]))
    mat.append(row)
cols=["时间\\到药材中心的距离"]+[str(d) for d in d_fix]+["药材表面"]
df4=pd.DataFrame(np.column_stack([out_t, np.round(np.array(mat),4)]), columns=cols)
p4=os.path.join(RES,"result4.xlsx")
df4.to_excel(p4, sheet_name="水分浓度", index=False)
from openpyxl import load_workbook
wb=load_workbook(p4)
for ws in wb.worksheets:
    for row in ws.iter_rows(min_row=2,min_col=2):
        for cc in row:
            if isinstance(cc.value,(int,float)): cc.number_format="0.0000"
wb.save(p4)
print("result4.xlsx: %d 行 x %d 列 (时间 %.0f~%.0f s, 步长60s; 距离0~1.1cm + 药材表面)"%(
    df4.shape[0],df4.shape[1],out_t[0],out_t[-1]))
h_rows=[6,12,18,24,30,36,42,48]
d_tab=[0.0,0.5,1.0]
rows=[]; labels=[]
for hh in h_rows:
    if hh*3600 < tend:
        k=int(np.argmin(np.abs(ts-hh*3600.0))); R=Rs[k]; c=Cs[k]
        row=[float(np.interp((d/100.0)/R, xi, c)) for d in d_tab]+[float(c[-1])]
        rows.append(np.round(row,4)); labels.append("%d"%hh)
k=-1; R=Rs[k]; c=Cs[k]
row=[float(np.interp((d/100.0)/R, xi, c)) for d in d_tab]+[float(c[-1])]
rows.append(np.round(row,4)); labels.append("%.2f"%(tend/3600))
T6=pd.DataFrame(rows,index=labels,columns=["0","0.5","1.0","药材表面"])
with pd.ExcelWriter(os.path.join(TAB,"\u88686.xlsx"),engine="openpyxl") as w:
    T6.to_excel(w,sheet_name="表6 水分浓度")
with open(os.path.join(TAB,"\u88686.txt"),"w",encoding="utf-8") as f:
    f.write("表6 药材烘干过程的水分浓度
        (kg/kg，考虑收缩)\n时间/h\\到药材中心的距离/cm\t0\t0.5\t1.0\t药材表面\n")
    for lab in labels: f.write(lab+"\t"+"\t".join("%.4f"%v for v in T6.loc[lab])+"\n")
    f.write("\n注：最后一行为烘干结束时间 %.2f h（各处水分浓度均 < 0.15 kg/kg）；\n"% (tend/3600))
    f.write(" 药材半径由 2.000 cm 收缩至 %.3f cm，故最终表面位于 %.3f cm 处。\n"%(Rs[-1]*100,Rs[-1]*100))
print(T6.to_string())
print("表6 written")
fig,ax=plt.subplots(1,3,figsize=(16,4.4))
for hh in [6,12,24,36,48]:
    k=int(np.argmin(np.abs(ts-hh*3600.0))); ax[0].plot(xi*Rs[k]*100,Cs[k],label="%dh"%hh)
ax[0].plot(xi*Rs[-1]*100,Cs[-1],'k--',lw=2,label="结束 %.2fh"%(tend/3600))
ax[0].axhline(0.15,color='r',ls=':',lw=1,label="目标0.15")
ax[0].set_xlabel("到中心距离 (cm)"); ax[0].set_ylabel("水分浓度 (kg/kg)");
    ax[0].set_title("问题4：收缩下的水分径向分布"); ax[0].legend(fontsize=8)
ax[1].plot(ts/3600,Cs[:,0],label="中心")
```

```
    ax[1].plot(ts/3600,Cs[:,N_FIN//2],label="r=0.5R")
    ax[1].plot(ts/3600,Cs[:,-1],label="表面")
    ax[1].axhline(0.15,color='r',ls=':',lw=1,label="目标0.15")
    ax[1].axvline(tend/3600,color='gray',ls='--',lw=1,label="结束 %.2fh"%(tend/3600))
    ax[1].set_xlabel("时间 (h)"); ax[1].set_ylabel("水分浓度 (kg/kg)");
         ax[1].set_title("问题4: 水分随时间变化"); ax[1].legend(fontsize=8)
    ax[2].plot(t2/3600,R2,'o-',ms=3,label="附件2 半径")
    ax[2].axvline(tend/3600,color='gray',ls='--',lw=1,label="烘干结束 %.2fh"%(tend/3600))
    ax[2].set_xlabel("时间 (h)"); ax[2].set_ylabel("半径 (cm)"); ax[2].set_title("药材半径随时间收缩");
         ax[2].legend(fontsize=8)
    fig.tight_layout(); fig.savefig(os.path.join(FIG,"q4_result.png"),dpi=140); plt.close(fig)
    print("figure written")
```